AIM :

To write a Python program to implement the A* algorithm for finding the shortest path.

ALGORITHM :

1. Start the program.
2. Create a graph with nodes and edge costs.
3. Assign heuristic values to each node.
4. Initialize the priority queue with the starting node.
5. Calculate the cost using the formula:

f(n) = g(n) + h(n)

6. Select the node with the lowest f(n) value.
7. Update the cost and parent of neighbouring nodes.

8. Repeat until the goal node is reached.
9. Print the shortest path and total cost.
10. Stop the program.

PROGRAM :

prgm 10.py - C:/Users/nivet/AppData/Local/Programs/Python/Python314/prgm 10.py (3.14.6)

File Edit Format Run Options Window Help

```
import heapq

def astar(graph, heuristic, start, goal):
    queue = [(heuristic[start], start)]
    cost = {start: 0}
    parent = {}

    while queue:
        f, current = heapq.heappop(queue)

        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()

            print("Path:", path)
            print("Cost:", cost[goal])
            return

        for neighbour, weight in graph[current]:
            new_cost = cost[current] + weight

            if neighbour not in cost or new_cost < cost[neighbour]:
                cost[neighbour] = new_cost
                parent[neighbour] = current
                priority = new_cost + heuristic[neighbour]
                heapq.heappush(queue, (priority, neighbour))

# Graph representation
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2)],
    'C': [('D', 1)],
    'D': [('G', 3)],
    'G': []
}

# Heuristic values
heuristic = {
    'A': 5,
    'B': 4,
    'C': 3,
    'D': 2,
    'G': 0
```

```
# Heuristic values
heuristic = {
    'A': 5,
    'B': 4,
    'C': 3,
    'D': 2,
    'G': 0
}

print("A* Algorithm:")
astar(graph, heuristic, 'A', 'G')
```

OUTPUT :

```
========== RESTART: C:/Users/nivet/AppData/Local/Proc
A* Algorithm:
Path: ['A', 'B', 'D', 'G']
Cost: 6
>>
```

RESULT :

Thus, the Python program to implement the A* algorithm was executed successfully.